Regex Matching Engine V2

A From-Scratch Guide to How the Project Works

A beginner-first technical report: no previous knowledge of compiler design, regular expressions, finite automata, Python, APIs, or web dashboards is assumed.

Team member	Registration number
Aprark	24BCI0052
Aayushi	24BCE0481
Shrey	24BCE2454

What this report does
It starts with the meaning of a character and a string, builds the necessary compiler-design ideas one at a time, follows a pattern through every project stage, explains the actual source modules and dashboard, and ends with exact run commands, measured behavior, safety limits, testing, glossary, and limitations.

Contents

Part	What the reader will learn
1. Foundations	Characters, strings, languages, regex, compilers, and why this is a compiler-design project
2. The complete pipeline	What enters the system, every transformation, and what comes out
3. Parsing	Grammar, precedence, cursor-based reading, AST nodes, and diagnostics
4. NFA construction	States, transitions, epsilon, fragments, and Thompson's rules
5. DFA matching	Epsilon closure, move, subset states, laziness, caching, and full-string semantics
6. V2 features	Character classes, numeric repetition, symbolic input classes, and minimization
7. Project surfaces	Simulation, analysis, API, dashboard, and test suite
8. Evidence and operation	Measured comparison, safety budgets, files, commands, glossary, and limitations

How to read the notation

Notation	Plain-language meaning
'a'	One character: the lowercase letter a
abc	A string containing a, then b, then c
ε	The empty string: zero characters
∅	No possible state or result
q0, q1	Names used for automaton states
{q0, q2}	A set containing two states

Notation	Plain-language meaning
→	The next transformation or transition

Important scope
This engine performs full-string matching. The complete input must fit the complete pattern. It is not a substring search tool and it does not use Python's built-in regular-expression engine to decide matches.

1 Foundations: the ideas needed before the code

1.1 Characters, strings, alphabets, and languages

A character is one unit of text, such as a, 7, ?, or a Unicode symbol. A string is an ordered sequence of characters. The empty string contains no characters and is written ε. The length of ε is zero. An alphabet is the set of characters allowed in a discussion. A language is a set of strings. In compiler design, a language does not mean English or Hindi; it means precisely which strings are considered valid.

Term	Example	Meaning
Character	a	One text symbol
String	abba	Four characters in order
Empty string	ε	A valid string of length zero
Alphabet	{a,b}	Only a and b are being considered
Language	{a, ab, abb, ...}	A selected set of strings

1.2 What is a regular expression?

A regular expression, shortened to regex, is a compact description of a language. The pattern a*b describes every string containing zero or more a characters followed by exactly one b. Therefore b, ab, aaab, and aaaaab belong to this language, while a, ba, and aaba do not.

Pattern part	Name	Plain meaning	Example accepted text
a	literal	Match exactly a	a
.	wildcard	Match any one character	x

Pattern part	Name	Plain meaning	Example accepted text
ab	concatenation	Match a and then b	ab
a|b	alternation	Match a or b	b
a*	Kleene star	Zero or more a characters	aaa
a+	plus	One or more a characters	a
a?	optional	Zero or one a	ε or a
(ab)	group	Treat ab as one unit	ab
[a-z]	class range	One lowercase a through z	m
[^0-9]	negated class	One character that is not a digit	Q
a{2,4}	numeric repeat	Two, three, or four a characters	aaa

1.3 What is a compiler?

A compiler translates one representation into another while preserving meaning. A programming-language compiler may translate source code into machine code. This project translates a regex from human-readable syntax into machine-executable automata. The destination is not CPU instructions; it is a network of states and transitions that can consume input characters and decide MATCH or NO MATCH.



Why this counts as compiler design
It has a source language, grammar, parser, intermediate representation, code-like lowering stages, execution model, diagnostics, resource limits, and correctness tests—the same categories found in larger compilers, on a deliberately smaller language.

2 The complete system from input to result

2.1 Inputs and outputs

The main match operation receives two strings: a pattern and an input. For example, pattern (a|b)*abb and input aabb. The output contains a Boolean decision, structured error data if compilation failed, the number of lazy-DFA states explored, and a bounded transition trace for the interface.

Stage	Receives	Produces	Actual module
Parse	Pattern text	Abstract syntax tree	regex_engine/parser.py
Construct	Syntax tree	Thompson NFA	regex_engine/nfa.py
Prepare	NFA	Lazy DFA start state	regex_engine/dfa.py
Execute	DFA + input	MATCH or NO MATCH	regex_engine/matcher.py
Explain	Pattern + input	NFA/DFA step table	regex_engine/simulation.py
Analyze	Workload	Timing and size metadata	regex_engine/analysis.py
Serve	JSON requests	JSON responses + dashboard	server/main.py and web/

2.2 A running example

We will repeatedly use (a|b)*abb. Read it left to right as: choose a or b, repeat that choice zero or more times, and finally require a, then b, then b. It describes all a/b strings ending in abb. The input aabb matches because it ends in abb. The input abab does not.

Layer	Representation of the same meaning
Source pattern	(a|b)*abb
Informal meaning	Any a/b prefix, followed by abb
AST shape	Concat(Star(Union(a,b)), a, b, b)
NFA meaning	Several possible active states connected by symbol and ε transitions
DFA meaning	One current subset of NFA states after each input character
Result for aabb	MATCH, because the final subset contains the NFA accept state

2.3 Compile once, match repeatedly

compile_pattern is cached for up to 256 distinct pattern strings. The first use parses and constructs the automata. Reusing the same pattern can reuse that compiled object. A re-entrant lock protects the shared lazy-DFA cache so concurrent requests do not mutate it unsafely. This cache is an optimization; it does not change the language accepted by the pattern.

Two uses of the word state
A parser state means where the parser cursor is. An automaton state means a node in an NFA or DFA. They are related only by the general idea of remembering the current situation.

3 Parsing: turning symbols into structure

3.1 Why parsing is necessary

The text a|bc is ambiguous unless precedence rules are defined. This project gives repetition the highest precedence, concatenation the next precedence, and alternation the lowest. Therefore a|bc means a or (b followed by c), not (a or b) followed by c. Parentheses override the normal grouping.

Priority	Operation	Example	Interpretation
Highest	Postfix repeat	ab*	a followed by zero or more b
Middle	Concatenation	abc	a, then b, then c
Lowest	Alternation	a|bc	a, or the complete string bc
Explicit	Parentheses	(a|b)c	a or b first, then c

3.2 The grammar in plain language

```
union  := concat ('|' concat)*
concat := repeat*
repeat := atom postfix?
atom   := literal | '.' | class | '(' union ')' | escaped-character
postfix:= '*' | '+' | '?' | numeric-repeat
```

The := symbol means ‘is formed from’. The final * in the grammar is descriptive notation meaning repeat the grammar piece, not the star token being parsed. union first asks for a concatenation and then absorbs additional | branches. concat collects adjacent repeated atoms. repeat attaches at most one postfix operator to one atom. atom recognizes the smallest units.

3.3 Cursor-based reading

The Parser object stores the complete pattern and a cursor position. peek observes the current character without consuming it. advance returns the current character and moves the cursor one place. Each grammar function consumes exactly the characters belonging to its construct. At the end, parse verifies that no unexpected character remains.

- Loops parse long flat concatenations and alternations without growing Python’s call stack.
- Recursion is used only when parentheses create real nesting.
- Group nesting is counted and rejected above 200 levels.

- The top-level empty pattern is legal and becomes an Empty AST node; an empty group or alternation branch is an error.

3.4 Abstract syntax tree (AST)

An AST keeps meaning and removes punctuation that is no longer needed. Each node is a data object. Literal stores one character. AnyChar represents the dot wildcard. CharClass stores explicit characters plus a negated flag. Concat and Union store ordered child lists. Star, Plus, Optional, and Repeat wrap one inner node.

AST node	Children/data	Meaning
Empty	none	Accept only ε
Literal	character	Accept exactly that character
AnyChar	none	Accept any one character
CharClass	characters + negated	Accept membership or non-membership
Concat	parts	Accept parts in sequence
Union	branches	Accept any one branch
Star / Plus / Optional	inner	Apply repetition rule
Repeat	inner, minimum, maximum	Apply numeric repetition

3.5 Parsing the running example step by step

Cursor sees	Parser action	AST result so far
(	Start a group and recursively parse its contents	pending group
a	Create Literal('a')	a
|	Finish first concat and start another union branch	Union(a, ...)
b	Create Literal('b')	Union(a,b)
)	Close the group	Union(a,b)

Cursor sees	Parser action	AST result so far
*	Wrap the group in Star	Star(Union(a,b))
a	Append another literal to concatenation	..., a
b	Append b	..., a, b
b	Append final b	..., a, b, b
end	Return the complete root	Concat(Star(Union(a,b)),a,b,b)

3.6 Character classes

Inside brackets, ordinary characters are collected into a set. A hyphen between two valid endpoints expands an inclusive range, so [a-c] becomes {a,b,c}. A leading ^ after [negates membership. Escapes allow a class metacharacter to be literal. A descending range such as [z-a], an unclosed class, or an empty class is rejected with a typed diagnostic.

3.7 Numeric quantifiers

Form	Minimum	Maximum	Meaning
{m}	m	m	Exactly m copies
{m,n}	m	n	Between m and n inclusive
{m,}	m	unbounded	At least m copies

The parser checks for missing numbers, missing closing braces, a maximum smaller than the minimum, and counts above 1,000. Only one postfix quantifier is allowed, so a** or a+? is rejected rather than assigned an accidental meaning.

3.8 Structured errors instead of crashes

RegexSyntaxError carries a stable code, human message, start and end positions, and an optional hint. diagnostic extracts a short context window and creates a caret pointer. RegexLimitError represents a safe refusal when an input or automaton would exceed a budget. The public match function catches both and returns a MatchResult; malformed user input does not leak a Python traceback.

```
Pattern: a{4,2}
Code: reversed_repeat_bounds
Context: a{4,2}
         ^^^^^
Meaning: the upper bound cannot be smaller than the lower bound
```

4 Finite automata: machines made of states

4.1 The four ingredients

A finite automaton is a small abstract machine. It has a finite set of states, one start state, some accepting states, and labeled transitions. It reads the input from left to right. A transition answers: if the machine is here and sees this character, where may it go next?

Ingredient	Question it answers
State	What situation is the machine currently remembering?
Start state	Where is the machine before reading any character?
Transition	Where can it move after consuming a character?
Accept state	Is the complete consumed input valid?

4.2 NFA versus DFA

Property	NFA	DFA
Full name	Nondeterministic finite automaton	Deterministic finite automaton
Next choices	May have zero, one, or many	At most one for a character
ε transitions	Allowed	Not used during matching
Runtime representation	Set of active states	One DFA state
Ease of construction	Very convenient from regex syntax	Convenient and fast to execute

Nondeterministic does not mean random. It means the mathematical machine may follow all valid choices at once. Software represents those simultaneous possibilities as a set. A DFA state can therefore be understood as one frozen set of NFA states.

4.3 Epsilon transitions

An ε transition moves without consuming an input character. It is wiring used to combine fragments. If q0 has an ε transition to q1, the NFA may be in q1 immediately while the input cursor stays in the same place. Epsilon closure repeatedly follows ε transitions until no new state can be reached.

Common misconception
ε is not a visible letter in the input. It represents consuming nothing. An ε transition can change the machine state but

cannot advance the input position.

4.4 Thompson construction and fragments

The project uses Thompson construction. Every AST node is compiled into a fragment containing a start state and an accept state. Larger fragments are built by connecting smaller fragments with ε transitions. This gives a simple, systematic rule for every supported regex operator.

4.5 Thompson rules, one operator at a time

AST form	Construction rule	Why it works
Literal a	Two states; a transition from start to accept	Exactly one a must be consumed
Wildcard/class	Two states; symbolic transition	Exactly one matching character is consumed
Concat XY	Connect X accept to Y start with ε	Y begins only after X finishes
Union X|Y	New start fans out to X/Y; accepts join	Either branch may succeed
Star X*	May skip X; after X may loop or exit	Allows zero or more copies
Plus X+	Must enter X; then may loop or exit	Allows one or more copies
Optional X?	May enter X or skip directly	Allows zero or one copy
Repeat	Build required copies, optional copies, and possibly a trailing star	Reduces numeric syntax to the primitive rules

4.6 Internal NFA representation

NFAState stores a mapping from a transition symbol to target state IDs plus a separate epsilon target set. A literal symbol is a character. The wildcard uses an ANY sentinel. A character class uses an immutable CharSet object. symbol_matches centralizes the question ‘does this symbolic edge accept this input character?’ State IDs are allocated by the builder and are stable within that constructed NFA.

4.7 Why the NFA grows linearly

Each primitive and operator adds a bounded number of states around its children. A flat pattern therefore produces an NFA whose size grows roughly in proportion to the pattern representation. Numeric repeats deliberately copy the inner fragment, which is why repeat counts are capped at 1,000 and total NFA states are capped at 50,000.

4.8 Worked miniature: a|b

```
new start --ε--> a-start --a--> a-accept --ε--> new accept
       \--ε--> b-start --b--> b-accept --ε--/
```

Before reading input, epsilon closure activates the new start and both branch starts. Reading a reaches the a-accept, then closure reaches the final accept. Reading b follows the other branch. Reading c reaches no symbol transition and therefore cannot match.

5 From NFA choices to deterministic matching

5.1 The move operation

move receives a set of active NFA states and one input character. It inspects outgoing symbol transitions from every active state, tests each symbol with symbol_matches, and unions all matching targets. It does not follow epsilon transitions; that is handled by epsilon_closure immediately afterward.

5.2 One character-processing cycle



1. Start with epsilon_closure({NFA start}).
2. For the next input character, compute all symbol-transition targets.
3. Expand those targets through every reachable ε transition.
4. Treat the resulting frozen set as the next DFA state.
5. After the entire input, accept only if that set contains the NFA accept state.

5.3 Subset construction

Classic subset construction can build every reachable NFA-state subset in advance. The theoretical maximum for an N-state NFA is 2^N subsets. Many of those subsets are never needed for a real input. This project therefore performs subset construction lazily: it creates a DFA state only when matching or graph exploration asks for a transition that reaches it.

5.4 LazyDFA internals

Each DFAState has an integer ID, a frozenset of NFA state IDs, and an is_accept flag. LazyDFA keeps a dictionary from frozenset to the canonical DFAState object and a transition cache. The start DFA state is the epsilon closure of the NFA start. A missing move returns None, which acts as immediate rejection rather than storing a permanent dead state in normal matching.

5.5 Character signatures and cache reuse

A transition cannot be cached only by the literal input character when wildcard and character classes exist. Instead, character_signature describes how a character behaves against the NFA's symbolic transitions. Characters with the same signature share one cached transition. For example, many characters not explicitly mentioned by a pattern can share the same 'other' behavior.

5.6 Full-string matching

The engine begins before the first character, consumes characters in order, and checks acceptance only after all characters are consumed. If a transition becomes impossible, it rejects early. This is equivalent to

asking whether the input belongs to the complete language of the pattern, not whether any substring happens to fit.

5.7 The public match algorithm in exact order

Step	Operation	Possible result
1	Look up or compile the pattern	CompiledPattern or structured syntax/limit error
2	Reject input above 2,000,000 characters	Limit result
3	Lock the shared compiled object	Safe access to lazy cache
4	Set current state to DFA start	Initial trace row
5	For each character, call dfa.step	Next state, None, or state-limit error
6	Record at most 500 trace rows	Bounded UI data
7	Test final state's is_accept flag	MATCH or NO MATCH
8	Return counts and diagnostics	MatchResult data object

5.8 A trace of (a|b)*abb against aabb

The exact numeric state IDs depend on the constructed NFA, but the semantic progression is stable. The subset after each character remembers every suffix-progress possibility that remains alive.

Input position	Character	Conceptual situation	Accept?
0	—	At start; star may repeat or finish; final abb may begin	No
1	a	Prefix loop remains possible; suffix has matched a	No
2	a	Prefix loop remains possible; newest suffix candidate has matched a	No
3	b	A candidate suffix has matched ab	No

Input position	Character	Conceptual situation	Accept?
4	b	A candidate suffix has matched abb	Yes

Why multiple possibilities matter
After reading an a, the engine cannot know whether that a belongs to the arbitrary prefix or is the first character of the final abb. The NFA keeps both possibilities. The corresponding DFA subset packages them into one deterministic runtime state.

5.9 Thread safety and the compilation cache

lru_cache(maxsize=256) avoids rebuilding frequently used patterns. Since the LazyDFA inside a compiled pattern learns new states and transitions, CompiledPattern owns an RLock. Matching the same compiled object is serialized around that mutation. Simulations create a fresh LazyDFA when they need deterministic isolated state numbering.

6 V2 language and automata features

6.1 Character classes are symbolic edges

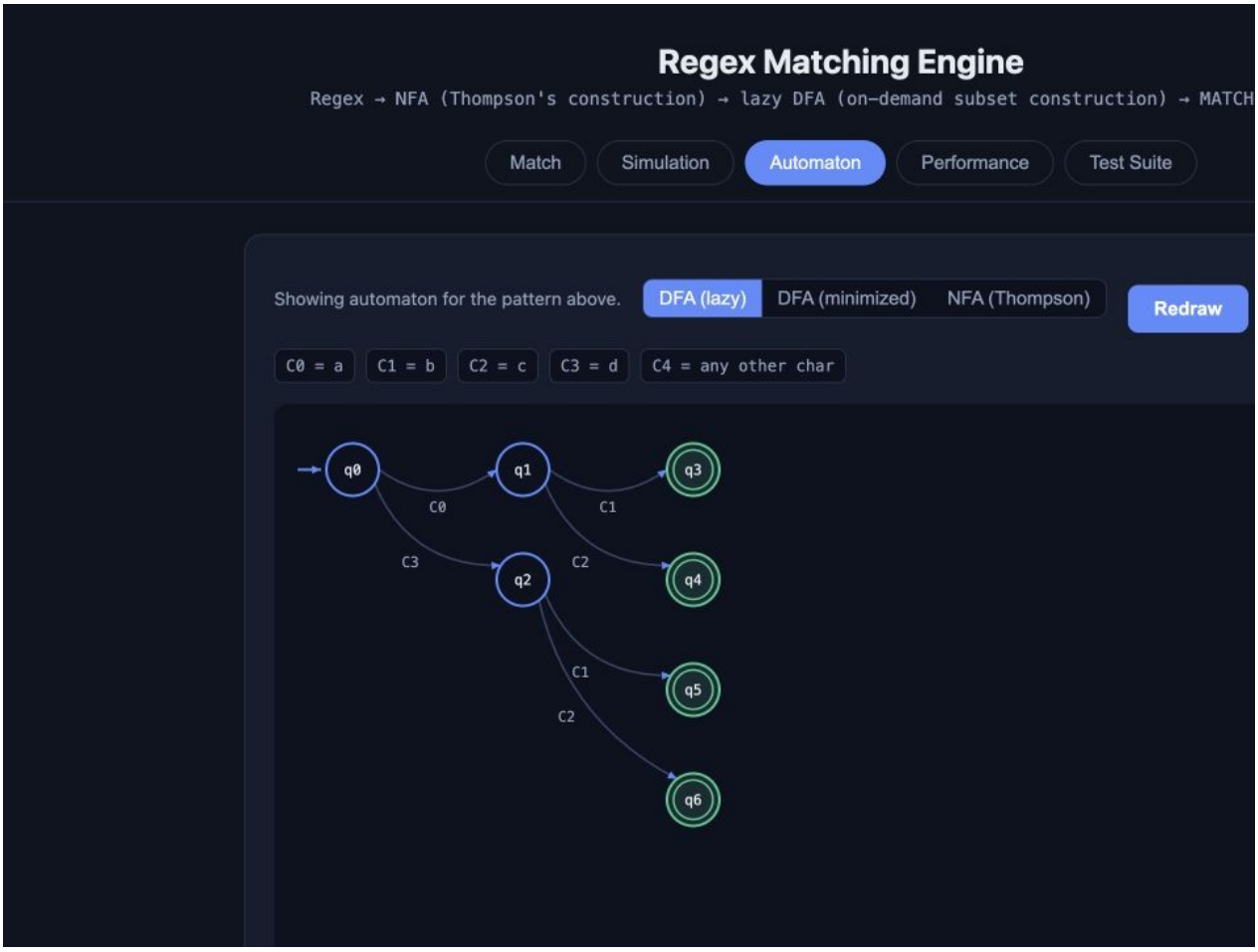
A class such as [A-Za-z] does not create one NFA branch for every letter. It becomes one CharSet transition holding the accepted set. A negated class reverses membership: [^0-9] accepts a character if it is not in the stored digit set. This compact representation is important for graph size and Unicode behavior.

6.2 Numeric repetition is compiled, not interpreted at runtime

Repeat nodes are lowered during NFA construction. For {m}, the builder concatenates m copies. For {m,n}, it adds m required copies and n−m optional copies. For {m,}, it adds m required copies followed by a star copy. The matcher therefore needs no special counter; it executes the resulting ordinary automaton.

6.3 Symbolic input equivalence classes

For complete-DFA graphing and minimization, the engine groups characters that cause identical NFA transition behavior. Each group has a representative character and a readable label. The dashboard draws compact edge labels C0, C1, and so on, then shows the full meaning in a legend. One extra representative covers characters not explicitly mentioned by the pattern.



Lazy DFA graph: compact C-labels keep class-heavy transitions readable; the legend expands their meanings.

7 DFA minimization from first principles

7.1 What minimization means

Two DFA states are equivalent if no possible future input can make one accept and the other reject. Equivalent states remember different histories but have identical future behavior. Minimization merges them, producing a smaller DFA that recognizes exactly the same language.

7.2 Why a complete DFA is required

The minimizer needs a defined destination for every state and every symbolic input class. It therefore constructs a complete reachable DFA and includes the empty subset as a dead state. The dead state represents ‘the match has already become impossible’; every later character remains dead. Normal lazy matching does not need to store this state, but formal minimization does.

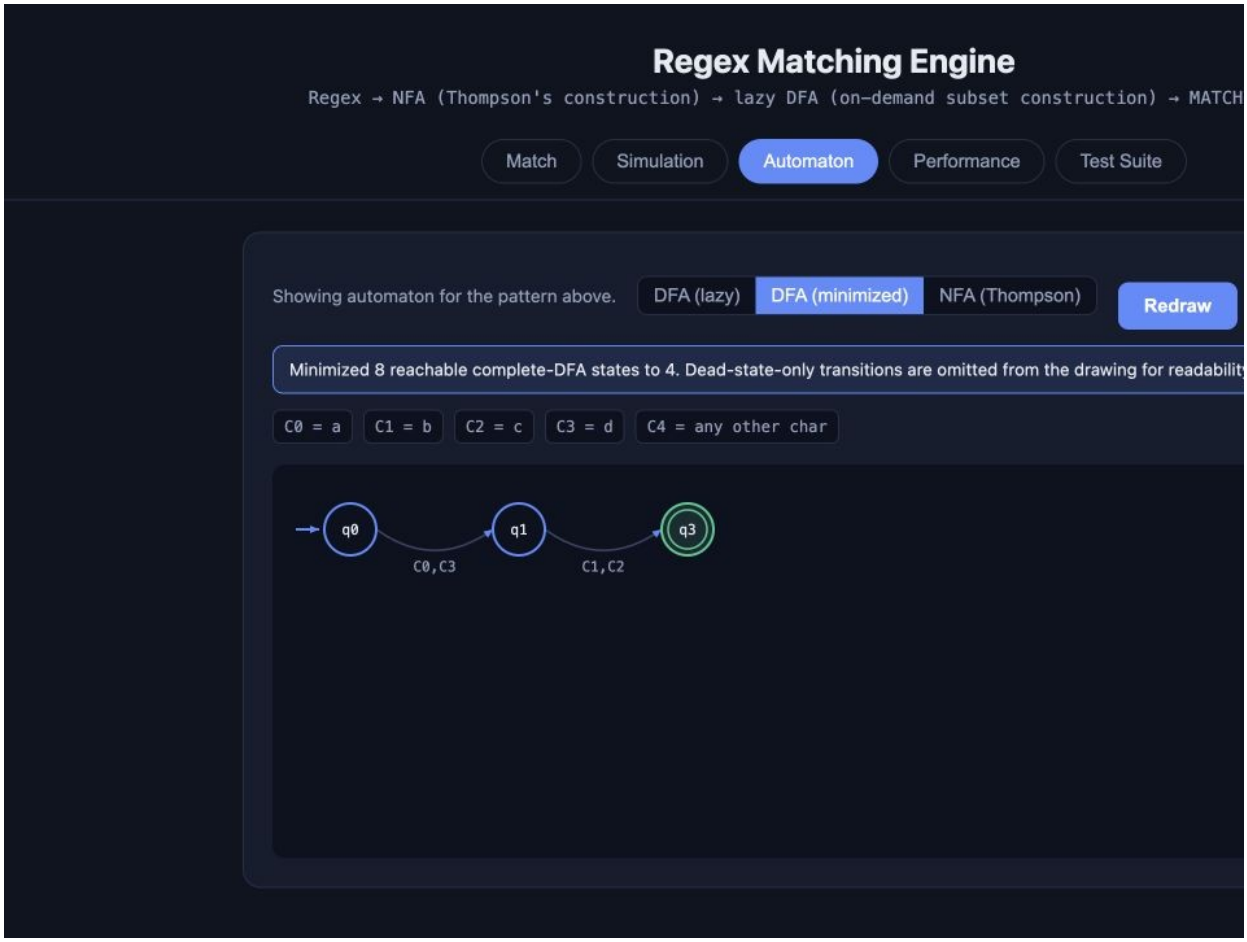
7.3 Partition refinement

1. Build every reachable complete-DFA state over the symbolic input classes.
2. Split states into two initial groups: accepting and non-accepting.
3. For each state, record which current group each input class leads to.
4. If two states in one group have different destination-group signatures, split the group.
5. Repeat until no group changes. Each remaining group becomes one minimized state.

7.4 Demonstration workload

The project’s live demonstration uses (ab|ac|db|dc). Before minimization, the complete reachable DFA has 8 states. Partition refinement merges equivalent suffix situations, leaving 4 states. The reduction is 4 states, or 50%. The language remains exactly the four strings ab, ac, db, and dc.

Input	Expected	Reason
ab	MATCH	Listed branch
ac	MATCH	Listed branch
db	MATCH	Listed branch
dc	MATCH	Listed branch
aa	NO MATCH	Second character is invalid
abc	NO MATCH	Full-string match rejects the extra c
	NO MATCH	Two characters are required



Minimized DFA view. Dead-state-only edges are omitted from the drawing for readability, while the minimization calculation still includes the dead state.

8 Simulation: making invisible execution visible

8.1 NFA simulation

NFA simulation begins with an epsilon closure. For each input character it reports the previous active states, symbol transitions actually taken, direct move targets, the next epsilon-closed active set, and whether the accept state is active. States and transitions are sorted so repeated runs are deterministic and easy to compare.

8.2 DFA simulation

DFA simulation starts from a fresh LazyDFA, records the current DFA ID and its underlying NFA subset, then records one row per consumed character. A missing transition is shown as a dead step. The result is MATCH only if all input was consumed and the final state is accepting.

NFA column	Meaning
before_states	Epsilon-closed set before consuming the character
transitions	Matching labeled NFA edges taken

NFA column	Meaning
move_states	Direct symbol-edge targets
active_states	Move targets after epsilon closure
accept	Whether the NFA accept is active

DFA column	Meaning
from_state	DFA ID before the character
dfa_state	DFA ID after the character, or null
nfa_states	The subset represented by the new DFA state
dead	True when no transition exists
accept	Whether the represented subset contains NFA accept

8.3 Simulation budgets

The engine returns at most 500 displayed steps and at most 200 transition records per NFA step. Matching may continue after the display limit; only explanatory output is truncated. This prevents a huge input from generating a huge browser table or response object.

8.4 Match trace versus simulation

MatchResult contains a lightweight DFA trace for normal use. The simulation module provides richer pedagogical data and supports both NFA and DFA modes. Keeping explanation separate from the minimal match path makes the normal API simpler while allowing the dashboard to teach the algorithm.

9 Performance analysis and measured evidence

9.1 What the built-in analyzer measures

analysis.py deliberately creates a fresh AST and NFA, attempts complete-DFA minimization, creates a fresh lazy DFA, performs five warm-up runs, and then measures 1 to 100 repeated matches with time.perf_counter_ns. It reports minimum, median, 95th percentile, and maximum time, plus environment metadata and a SHA-256 checksum of the workload.

Field	Why it exists
Warm-ups	Populate the lazy transition cache before timed matching

Field	Why it exists
Repeats	Reduce dependence on one noisy timing sample
Median	Describe the typical measured run
p95	Show a high-end value without using only the single maximum
Checksum	Prove which exact pattern/input pair was measured
Environment	Record Python, platform, and machine
State counts	Connect runtime observations to automaton size

9.2 Fair V1 versus V2 protocol

The project review benchmark checked out V1 commit 09de2a6 in an isolated copy and ran V1 and V2 with the same Python executable, machine, workload strings, warm-ups, repeat count, and process-level measurement method. Only syntax supported by both versions was used. V2-only character classes and numeric quantifiers were measured separately rather than pretending V1 could run them.

Workload	V1 median ms	V2 median ms	V2/V1	DFA states V1/V2	Peak RSS MiB V1/V2
suffix-language	0.453	1.563	3.45×	4/4	27.31/27.39
wildcard-scan	0.410	1.866	4.55×	4/4	27.45/27.39
alternation-loop	0.417	1.671	4.01×	7/7	27.44/27.41

On these three shared-syntax workloads, V2 matching was 3.45× to 4.55× slower while NFA/DFA state counts and peak process memory remained effectively flat. The source-grounded explanation is that V2 computes generalized symbolic-class signatures for characters. This is an honest measured limitation, not a claim that V2 is universally slower or that the difference applies to every workload.

10 Server and dashboard

10.1 Backend architecture

FastAPI receives browser requests, validates field sizes with Pydantic models, calls the engine, converts data classes to JSON-compatible dictionaries, and serves the static dashboard. The frontend is plain HTML, CSS, and JavaScript; there is no separate build step. StaticFiles mounts web/ at the root path.

Endpoint	Method	Purpose
/api/match	POST	Compile and full-match one pattern/input pair
/api/automaton	POST	Return NFA, lazy-DFA, or minimized-DFA graph data
/api/simulate	POST	Return detailed NFA or DFA steps
/api/analyze	POST	Return bounded reproducible metrics
/api/tests	GET	Run and return the predefined dashboard suite
/	GET	Serve the browser dashboard

10.2 The five dashboard tabs

Tab	User action	What happens internally
Match	Enter regex and input	Calls /api/match; shows decision, diagnostics, trace
Simulation	Choose NFA or DFA	Calls /api/simulate; fills a step table
Automaton	Choose graph kind	Calls /api/automaton; draws states, arrows, and class legend
Performance	Choose repeat count	Calls /api/analyze; displays timing and size data
Test Suite	Run built-in cases	Calls /api/tests; compares expected and actual results

10.3 Graph generation

NFA graph data combines parallel labels between the same source and target. Lazy-DFA graph generation performs breadth-first exploration over symbolic input classes and uses C-labels. The minimized graph includes reduction metadata but omits the dead state and edges leading only to it from the picture. Rendering caps are 500 NFA states and 300 DFA states; truncation is reported rather than silently exhausting the page or browser.

10.4 Request validation

API models cap patterns at 20,000 characters and inputs at 2,000,000 characters before the engine runs. The regex parser, NFA builder, DFA builder, simulation, analysis, and rendering paths add their own narrower budgets. Defense in depth means a missed check in one layer is less likely to become uncontrolled resource use.

11 Testing and correctness

11.1 What ‘correct’ means

Correctness requires more than returning MATCH for a few examples. The parser must assign the intended structure, malformed syntax must be rejected, Thompson fragments must be connected correctly, lazy DFA caching must preserve behavior, NFA and DFA simulations must agree, minimization must preserve the language, limits must fail safely, and the HTTP endpoints must serialize useful responses.

11.2 Automated test groups

Test file	Primary responsibility
test_parser.py	AST structure, escapes, malformed patterns, nesting and length
test_nfa.py	Literal/concat/union/star construction and linear size
test_dfa.py	Lazy materialization, cache reuse, dead transitions, signatures
test_matcher.py	End-to-end decisions, diagnostics, unknown input characters
test_limits.py	Explosion avoidance, long input, caps, never-crash behavior
test_simulation.py	Deterministic NFA/DFA steps and dead transitions
test_analysis.py	Bounded self-describing measurements
test_server.py	Graph, simulation, analysis, and label API contracts
test_v2_syntax_and_automata.py	Classes, repeats, minimization, cross-checks

11.3 Differential and preservation tests

For the supported subset, V2 syntax tests compare outcomes with Python re.fullmatch. This is differential testing: an independent mature implementation acts as an oracle for overlapping semantics. Separate tests compare minimized and lazy DFAs across inputs to verify that minimization changes state organization but not the accepted language.

11.4 Dashboard suite

server/test_suite.py defines 29 visible cases across valid inputs, V2 classes, V2 quantifiers, ordinary non-matches, boundaries, and malformed regexes. A malformed case passes when the engine reports an error instead of raising. The long-input case uses 10,000 a characters. This suite is demonstrative evidence in the UI; the complete pytest suite is the deeper engineering check.

Verified project state
The last full verification associated with this build reported 117 passing pytest tests. The guide-generation step does not alter engine source or test behavior; the final handoff should still include the exact command so the reader can rerun verification locally.

12 Safety budgets and graceful failure

Regex engines can consume extreme time or memory when patterns or outputs become enormous. This educational engine uses explicit, documented ceilings. Reaching a ceiling returns a RegexLimitError or truncation flag; it does not intentionally continue until the process crashes.

Resource	Limit	Reason
Pattern length	20,000 characters	Bound parsing and request size
Group nesting	200	Bound recursive group parsing
Character-class size	4,096 explicit characters	Bound stored class data
Numeric repeat count	1,000	Bound expansion during NFA construction
NFA states	50,000	Bound automaton memory
Lazy DFA states	20,000	Bound on-demand subset cache
Complete DFA for minimization	5,000 states	Bound exhaustive exploration
Symbolic alphabet classes	2,048	Bound transition columns
Input length	2,000,000 characters	Bound match work
Displayed simulation steps	500	Bound response and browser table
Transitions per simulation step	200	Bound detailed trace size
NFA/DFA graph nodes	500 / 300	Bound graph rendering

Resource	Limit	Reason
Analysis repeats	100	Bound benchmark duration

12.1 Limits versus syntax errors

A syntax error means the pattern has no valid interpretation in the supported language, such as an unmatched parenthesis. A limit error means the pattern may be meaningful but this implementation refuses its requested scale, such as 20,001 pattern characters. Keeping these categories separate helps users correct the right problem.

12.2 Why laziness is also a safety mechanism

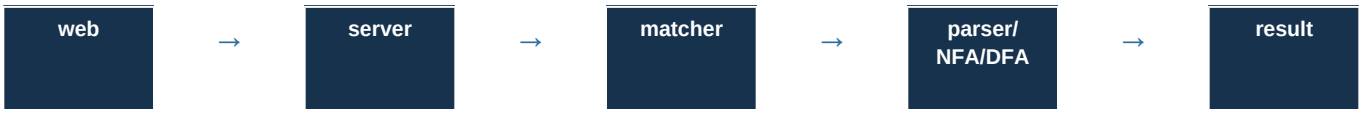
The power-set worst case belongs to complete subset construction. Normal matching materializes only states reached by actual characters, and stops immediately on a dead transition. This does not remove the theoretical worst case, so the 20,000-state cache cap remains necessary, but it makes ordinary workloads much more practical.

13 Project file map

Path	What it contains
regex_engine/parser.py	Grammar, AST node classes, cursor parser, syntax validation
regex_engine/nfa.py	Thompson construction, NFA data model, symbolic transitions
regex_engine/dfa.py	Epsilon closure, move, lazy DFA, input classes, minimization
regex_engine/matcher.py	Compilation cache, public match operation, MatchResult
regex_engine/simulation.py	Deterministic explanatory NFA and DFA traces
regex_engine/analysis.py	Bounded reproducible performance measurements
regex_engine/errors.py	Structured syntax diagnostics and limit errors
server/main.py	FastAPI request models, endpoints, graph extraction, static hosting
server/test_suite.py	29 visible dashboard cases
web/index.html	Five-tab page structure
web/app.js	API calls, tables, diagnostics, graph drawing

Path	What it contains
web/style.css	Dashboard layout and appearance
tests/	Unit, integration, safety, equivalence, and endpoint tests
docs/V2_SEMANTICS.md	Exact V2 semantics, budgets, and analysis protocol
scripts/review_benchmark.py	Fair isolated V1/V2 review benchmark
benchmarks/review_2026_10_27/	Raw data, environment metadata, and graphs
review/	Review report, presentation, demo guide, and this beginner guide

13.1 Dependency direction



The engine layer does not depend on the browser. The server imports the engine. The browser speaks JSON to the server. This separation allows pytest to verify core algorithms without launching a browser and allows another interface to reuse the engine later.

13.2 Data objects instead of loose dictionaries

Core structures use Python data classes: AST nodes, NFA states, DFA states, compiled patterns, diagnostics, and results. Data classes make fields explicit and support reliable conversion to dictionaries at the API boundary. Frozen data such as CharSet and syntax diagnostics cannot be accidentally mutated after creation.

14 Running the project from scratch

14.1 Requirements

- A terminal on the machine containing the project directory.
- Python and the project virtual environment at .venv.
- A modern browser for the dashboard.
- No Node.js build step and no external database are required.

14.2 Run the complete automated suite

```
cd /Users/shrey/Documents/compiler/project
PYTHONDONTWRITEBYTECODE=1 ./.venv/bin/python -m pytest -q -p no:cacheprovider
```

PYTHONDONTWRITEBYTECODE=1 prevents .pyc cache files during the check. -q requests concise output. -p no:cacheprovider prevents pytest's cache directory. A successful run ends with the passed-test count and no failures.

14.3 Start the dashboard

```
cd /Users/shrey/Documents/compiler/project
./.venv/bin/python -m uvicorn server.main:app --reload --port 8008
```

uvicorn runs the FastAPI application object named app in server/main.py. --reload restarts the server when source changes. --port 8008 selects the local TCP port. Keep this terminal open, then visit http://127.0.0.1:8008 in a browser. Stop the server with Control-C.

14.4 First manual walkthrough

1. In Match, enter (a|b)*abb and aabb. Expect MATCH.
2. Change the input to abab. Expect NO MATCH.
3. Enter a{4,2}. Expect a structured diagnostic pointing to the invalid bounds.
4. In Simulation, compare NFA and DFA rows for ab.
5. In Automaton, enter (ab|ac|db|dc), choose Minimized DFA, and confirm 8 reachable states before and 4 after.
6. In Performance, use the default 25 repeats and read the warm-up, timer, and environment metadata.
7. In Test Suite, run all 29 visible cases and check expected versus actual.

14.5 Run the reproducible review benchmark

```
cd /Users/shrey/Documents/compiler/project
./.venv/bin/python scripts/review_benchmark.py --help
```

Read the help before launching a benchmark because the script manages an isolated V1 comparison and output paths. Existing measured data is preserved under benchmarks/review_2026_10_27; do not overwrite it unless intentionally producing a new dated run.

15 Complete worked example: [A-Za-z]{2,4}[0-9]{2}

15.1 Human meaning

The first class accepts one English letter, uppercase or lowercase. {2,4} requires two through four such letters. The second class accepts one digit. {2} requires exactly two digits. Therefore Ab12 and code99 match; A12 has too few letters, codes99 has too many letters, and Ab1 has too few digits.

15.2 Parser output

```
Concat(
  Repeat(CharClass(A-Z and a-z), minimum=2, maximum=4),
  Repeat(CharClass(0-9), minimum=2, maximum=2)
)
```

15.3 NFA construction

The first Repeat becomes two required letter-class fragments followed by two optional letter-class fragments. The second Repeat becomes two required digit-class fragments. Concatenation joins the fragments with epsilon transitions. Each class remains a symbolic edge rather than expanding to dozens of literal branches.

15.4 Lazy-DFA execution for Ab12

Consumed	Required progress	Outcome
A	First required letter	alive
b	Second required letter; minimum reached	alive
1	Skip remaining optional letters; first required digit	alive
2	Second required digit	accepting after complete input

15.5 Why the graph legend matters

A literal edge label containing every accepted letter would overlap states and arrows. The graph instead labels the transition C0 and places the complete description in a legend. Characters that behave the same are grouped, so the graph represents the automaton accurately without turning the canvas into unreadable text.

15.6 Failure cases

Input	Result	Exact reason
A12	NO MATCH	Only one letter; minimum is two
codes99	NO MATCH	Five letters; maximum is four
Ab1	NO MATCH	Only one digit; exactly two required
Ab12x	NO MATCH	Extra trailing x; matching is full-string
12Ab	NO MATCH	Digit appears where a letter class is required

16 Glossary

Term	Beginner definition

Term	Beginner definition
Epsilon closure	All states reachable through zero or more ε transitions
Fragment	A start/accept pair used while constructing an NFA
Full-string match	A match requiring every input character to be consumed
Grammar	Rules defining which patterns are valid and how they group
Language	The set of all strings accepted by a pattern or automaton
Lazy construction	Creating a state only when it is needed
Literal	A regex atom matching one exact character
NFA	An automaton that may keep multiple possible states at once
Parser	The component translating token characters into an AST
Partition	A group of states currently considered equivalent
Precedence	Rules deciding which operation binds more tightly
Quantifier	An operator specifying how many copies may occur
Regex	A compact description of a regular language
State	A finite memory position in an automaton
Subset construction	Representing a set of NFA states as one DFA state
Symbolic transition	An edge labeled by a predicate such as wildcard or class
Thompson construction	A rule-based conversion from regex AST to NFA
Transition	A labeled movement from one state to another
Unicode	A standard assigning text characters consistent code points
Wildcard	The dot atom, matching any one character

17 Limitations

This is a focused educational regular-expression engine, not a complete replacement for production regex libraries. Its limitations are deliberate and should be stated clearly.

Limitation	Practical effect
No anchors (^ and $)	Matching is already full-string; anchor syntax is not supported
No capturing groups	Parentheses group syntax but do not return captured substrings
No backreferences	A later part cannot demand the exact text captured earlier
No lookahead or lookbehind	The engine cannot assert context without consuming it
No shorthand classes	Syntax such as \d, \w, and \s is not implemented
No case-insensitive or multiline flags	Matching behavior cannot be switched with flags
No substring search	The entire input must match; there is no search/find-all operation
Numeric repeat expansion	Large repeats enlarge the NFA, so counts are capped at 1,000
Theoretical state explosion remains	Lazy construction reduces unnecessary work but cannot abolish worst cases
Complete minimization is bounded	Patterns needing over 5,000 reachable DFA states are refused
Graphs are capped	Large automata are truncated at documented display limits
Trace output is capped	Only the first 500 explanatory steps are returned
Shared compiled DFA uses a lock	Concurrent matches of the same pattern serialize cache mutation
Measured V2 overhead	Shared-syntax benchmark medians were 3.45×–4.55× slower than V1
Educational UI	The dashboard is local and has no accounts, persistence, or deployment hardening

17.1 Final understanding

The project is best understood as a small compiler followed by a small virtual machine. The parser gives structure to regex text. Thompson construction lowers that structure into an NFA. Lazy subset construction turns only needed NFA-state sets into deterministic runtime states. The matcher executes those states over a complete input. Simulation exposes the hidden steps, minimization merges equivalent complete-DFA states, analysis records bounded measurements, and the dashboard connects every capability to a visible interface.

One-sentence explanation for a review

Our project compiles a supported regular expression into a Thompson NFA, creates equivalent DFA subset states only when needed, and consumes the entire input deterministically, while also providing simulation, formal DFA minimization, reproducible analysis, diagnostics, tests, and explicit safety limits.